

Polymorphism

Polymorphism means "many forms." In Java, it allows you to treat different child objects (`GasCar` , `ElectricCar`) as if they were their parent type (`Vehicle`) [sports]. This is powerful because it lets you write one piece of code that works automatically with any type of vehicle you create in the future. 

The Big Magic: Upcasting

Polymorphism allows you to declare a variable using the **Parent type**, but instantiate it using a **Child type**. 

java

```
// Declared as a Vehicle, but instantiated as a GasCar
Vehicle myRide = new GasCar("Blue");
```

Even though the variable type on the left is `Vehicle`, Java looks at the real object on the right (`GasCar`) at runtime and executes the `GasCar` version of your methods.

Polymorphism in Action (Code Example)

Let's look at how polymorphism allows us to manage all our cars inside a single array loop and pass them into a single function.

Why is this so useful?

Imagine you are building a video game with 100 different types of cars. Without polymorphism, you would have to write 100 separate functions to start a race (`startGasCarRace()`, `startElectricCarRace()`, etc.). With polymorphism, you write **one single function** that accepts `Vehicle`, and it handles all 100 car types perfectly.

What Prints to the Console?

```
--- Testing Loop Polymorphism ---
The gas engine revs! Fuel is injected into cylinders.
The electric motor hums instantly! Battery power surges.

--- Testing Function Polymorphism ---
The Red vehicle prepares... The gas engine revs! Fuel is injected into cylinders.
The Black vehicle prepares... The electric motor hums instantly! Battery power surges.
```

Polymorphism in Action (Code Example)

```

public class Main {
    public static void main(String[] args) {

        // 1. POLYMORPHISM IN ARRAYS
        // We create an array of type Vehicle. It can hold ANY child of Vehicle!
        Vehicle[] fleet = new Vehicle[2];

        fleet[0] = new GasCar("Blue");    // Upcasted automatically
        fleet[1] = new ElectricCar("White"); // Upcasted automatically

        System.out.println("--- Testing Loop Polymorphism ---");
        for (Vehicle v : fleet) {
            // Java automatically figures out which version of accelerate() to run!
            v.accelerate();
        }

        System.out.println("\n--- Testing Function Polymorphism ---");
        GasCar mustang = new GasCar("Red");
        ElectricCar tesla = new ElectricCar("Black");

        // 2. POLYMORPHISM IN FUNCTIONS
        // We can pass different object types into the exact same function
        startRace(mustang);
        startRace(tesla);
    }

    // This function accepts a generic Vehicle parent type
    public static void startRace(Vehicle v) {
        System.out.print("The " + v.color + " vehicle prepares... ");
        v.accelerate(); // Polymorphic method call
    }
}

```